

POCKETSMART AI

File 3 — Project Design

1. Design Objective

The PocketSmart AI design translates the requirements into a web-based architecture in which the user provides a budget and natural-language requirements, the application processes those inputs through an AI recommendation layer, and the interface presents the resulting recommendations.

2. High-Level Architecture

User → Web Interface → Application / Recommendation Layer → Gemini API → Recommendation Response → Web Interface

The architecture separates user interaction from AI processing so the application can validate input, construct an appropriate prompt, send the request to the Generative AI service, and format the response before displaying it.

3. Major Components

Component	Responsibility
User Interface	Collect budget, category, preferences, and other requirements; display recommendations.
Input Validation	Check that required fields and budget information are present and usable.
Recommendation Engine	Prepare the user's requirements and apply the application's recommendation logic.
Gemini AI Service	Interpret the request and generate natural-language recommendation content.
Response Formatter	Organize AI output into a readable recommendation format.

4. User Interaction Flow

Step	Process
1	User opens PocketSmart AI.
2	User enters a budget and selects or describes the required category.
3	User adds preferences and constraints in natural language.
4	Application validates the submitted information.
5	Recommendation logic prepares the AI request.
6	Gemini processes the request and generates recommendation content.
7	Application formats the response.
8	User views the recommendations and their explanations.

5. Logical Data Flow

Data	Direction / Purpose
Budget	User → Application → AI prompt; acts as the primary spending constraint.
Category	User → Application → AI prompt; identifies the shopping/planning domain.
Preferences	User → Application → AI prompt; provides qualitative requirements.

AI Request	Application → Gemini; combines relevant user requirements.
AI Response	Gemini → Application; contains generated recommendation content.
Displayed Result	Application → User; presents the processed recommendation.

6. Proposed Interface Design

The interface should prioritize clarity and minimal interaction. A practical layout is a main input area containing the budget and requirement fields, followed by a recommendation results area. Recommendations should be visually separated so users can scan them quickly.

7. AI Prompt Design

The AI request should provide the model with the user's budget, category, preferences, and relevant constraints. The prompt should instruct the model to keep recommendations aligned with the stated budget and explain the relevance of the suggestions.

8. Error Handling Design

Situation	Application Behaviour
Missing budget	Ask the user to provide a budget before generating recommendations.
Missing requirement	Ask the user to describe what they need.
Invalid input	Display a clear validation message and request corrected information.
AI/API failure	Display a user-friendly error without exposing internal credentials or technical details.
No useful recommendation	Ask for additional preferences or constraints.

9. Security Design

API credentials should be stored as environment variables rather than hard-coded into frontend source files. Requests to external AI services should be made through the application's controlled backend/service layer where applicable.

10. Design Summary

The Project Design establishes a modular PocketSmart AI architecture: a simple user-facing input layer, validation and recommendation logic, a Gemini-powered AI service, and a structured results layer. This design provides the foundation for the subsequent development and implementation phase.